

# Manual de Cucumber (BDD con Gherkin)

---

## Table of Contents

1. BDD: Behavior-Driven Development
  2. ¿Qué es Cucumber?
    - 2.1. Dependencias Maven
  3. Anatomía de un fichero `.feature`
  4. Cucumber Expressions
  5. Step definitions (glue)
  6. DataTables: una tabla para un único paso
  7. `Scenario Outline` + `Examples`: repetir un escenario con datos distintos
  8. `Background`: pasos comunes a toda la feature
  9. Hooks: `@Before` / `@After`
  10. Compartir estado entre clases de steps
    - 10.1. Campo estático compartido
    - 10.2. Inyección de dependencias por constructor ( `cucumber-picocontainer` )
  11. Casos de uso, abuso y mal uso (S-SDLC)
  12. Ejecutar los escenarios: JUnit Platform + Maven/Eclipse
    - 12.1. Dependencias necesarias
    - 12.2. `maven-surefire-plugin`
  13. Informes legibles
  14. Buenas prácticas (y errores comunes)
  15. Referencia rápida
- 

Manual de referencia rápida sobre Cucumber. No documenta un ejercicio concreto: recoge, organizados por concepto, los mecanismos de Cucumber que aparecen en los ejercicios `Ej12_Cucumber`, `Ej12_Cucumber_S-SDLC` y `Ej12_Cucumber_Selenium` del curso, con ejemplos tomados o adaptados de esos proyectos.

## 1. BDD: Behavior-Driven Development

BDD es una evolución de TDD que traslada el ciclo rojo-verde a un nivel más alto: en vez de escribir primero un test unitario contra una clase, se escribe primero un **escenario de comportamiento**, en un lenguaje que entiendan también las personas no programadoras (negocio, QA, producto), y luego se implementa el código hasta que ese escenario pasa.

Gherkin (el `Given / When / Then`) es solo la sintaxis; lo que hace que algo sea BDD de verdad son tres ideas de fondo:

- **Lenguaje ubicuo.** Los escenarios se escriben con el vocabulario del negocio ("cesta", "pedido", "atacante"...), no con detalles técnicos ("insertar fila en la tabla `PRODUCTOS`"). Así el mismo texto sirve como especificación para negocio y como test ejecutable para desarrollo.

- **Conversación antes que código** (la práctica de los "three amigos": negocio + QA desarrollo). Los escenarios deberían salir de una discusión sobre qué debe hacer el sistema **antes** de que exista una sola línea de implementación — no ser un desarrollador solo "traduciendo" a Gherkin lo que ya ha decidido construir.
- **Documentación viva**. Los `.feature` no son un test que se escribe y se olvida: quedan como documentación del comportamiento del sistema, actualizada por definición (si el comportamiento cambia y el escenario no se actualiza, el escenario falla).



Escribir primero los `.feature` y luego el código para que pasen es el mecanismo técnico de BDD (muchas veces llamado **outside-in** o **ATDD**, Acceptance Test-Driven Development), pero no basta por sí solo: si un único desarrollador escribe los escenarios sin la conversación con negocio, le falta la parte que le da sentido a Cucumber más allá de ser "otro framework de test". Los ejercicios de este curso, de hecho, se construyeron al revés (primero la aplicación, luego los `.feature` para describirla) — es útil para aprender la herramienta, pero no es BDD en el sentido estricto.

Ej12\_Cucumber\_S-SDLC sí ilustra la parte colaborativa: cada escenario técnico en `src/test/resources/proyecto/` tiene una versión gemela en `src/test/resources/requisitos/`, escrita en un nivel de abstracción distinto — sin números ni detalles de implementación, en lenguaje puramente de negocio:

| proyecto/abuso_cesta.feature (ejecutable) | requisitos/abuso_cesta.feature (legible por negocio)                    |
|-------------------------------------------|-------------------------------------------------------------------------|
| Given el precio de un teclado es -50.0    | Given un atacante que conoce el funcionamiento de la cesta de la compra |
| When Compro 1 teclados                    | When intenta añadir un producto con un precio o una cantidad inválidos  |
| Then la operación es rechazada            | Then el sistema rechaza la operación y no altera el importe a pagar     |

La columna de `requisitos/` no tiene glue code (no se ejecuta): es documentación pensada para que alguien de negocio pueda leerla y validar que describe el riesgo correctamente, sin necesitar saber qué es un `Producto` ni cuánto vale un teclado en el test.

## 2. ¿Qué es Cucumber?

Cucumber es el motor que ejecuta ficheros `.feature` escritos en Gherkin: parsea el texto, localiza el método Java (o de otro lenguaje) que implementa cada paso y lo invoca. No es un framework de aserciones — las aserciones dentro de los steps siguen siendo las de JUnit (o AssertJ, Hamcrest...); Cucumber solo aporta el "traductor" entre el texto en lenguaje natural y el código.

## 2.1. Dependencias Maven

```
<dependency>
  <groupId>io.cucumber</groupId>
  <artifactId>cucumber-java</artifactId>
  <version>7.34.2</version>
  <scope>test</scope>
</dependency>
<dependency>
  <groupId>io.cucumber</groupId>
  <artifactId>cucumber-junit-platform-engine</artifactId>
  <version>7.34.2</version>
  <scope>test</scope>
</dependency>
```

XML

`cucumber-java` trae las anotaciones para escribir los steps (`@Given`, `@When`, `@Then` ...); `cucumber-junit-platform-engine` registra Cucumber como un motor más de JUnit Platform, para que Maven (Surefire) y el IDE puedan descubrirlo y ejecutarlo igual que a JUnit Jupiter — ver Ejecutar los escenarios: JUnit Platform + Maven/Eclipse.

## 3. Anatomía de un fichero `.feature`

Feature: Autenticación de usuarios

GHERKIN

Como usuario de la aplicación  
Quiero identificarme con mis credenciales  
Para poder acceder al área de gestión de clientes

Scenario: Login correcto  
Given estoy en la página de login  
When introduzco mis credenciales  
And pulso el botón entrar  
Then se crea mi sesión  
And veo la página de inicio

- **Feature**: agrupa varios **Scenario**: relacionados. El texto libre justo debajo (`Como...` / `Quiero...` / `Para...`) no lo interpreta Cucumber — es solo documentación con el formato clásico de historia de usuario, para que quien abra el fichero entienda de un vistazo el porqué de la feature.
- Cada **Scenario**: es un caso concreto, con sus propios pasos **Given** / **When** / **Then** . **And** encadena pasos del mismo tipo que el anterior sin repetir la palabra clave (Cucumber ignora si pone **Given** , **When** , **Then** o **And** : empareja por el **texto** del paso, no por la palabra clave — ver Step definitions (glue)).
- Las líneas que empiezan por **#** son comentarios. Se usan mucho para documentar **por qué** un escenario no tiene un **Given** (ver `login_abuso.feature` , en DataTables: una tabla para un único paso y **Scenario Outline** + **Examples** : repetir un escenario con datos distintos), o para dejar constancia de un escenario descartado sin borrarlo del todo.

## 4. Cucumber Expressions

Los pasos pueden llevar valores variables, capturados con llaves. Cucumber convierte automáticamente al tipo Java del parámetro:

| Expresión | Ejemplo de uso                                                                                                                      |
|-----------|-------------------------------------------------------------------------------------------------------------------------------------|
| {int}     | @When("Compro {int} teclados") → Integer cantidad                                                                                   |
| {double}  | @Given("el precio de un teclado es {double}") → Double precio                                                                       |
| {string}  | @When("intento acceder directamente a {string}") → String recurso<br>(el texto debe ir entre comillas en el .feature : "<recurso>") |

Son más legibles que expresiones regulares equivalentes y cubren la mayoría de casos; si hiciera falta un tipo propio (por ejemplo, convertir directamente a un objeto `Cliente`), Cucumber permite definir un `@ParameterType` — no hay ningún ejemplo en estos proyectos, todos usan los tipos incorporados.

## 5. Step definitions (glue)

```
@Given("el precio de un teclado es {double}")
public void el_precio_de_un_teclado_es(Double precio) {
    this.precioTeclado = precio;
}
```

JAVA

- `@Given` / `@When` / `@Then` son, a efectos de Cucumber, **el mismo mecanismo**: lo único que importa para emparejar un paso con un método es que el texto coincida (ignorando mayúsculas del propio texto de la expresión). Por eso `login_abuso.feature` puede tener un `Then estoy en la página de login` que en realidad ejecuta un método anotado con `@Given`, sin que Cucumber se queje.



Que Cucumber no distinga la palabra clave es cómodo para reutilizar pasos, pero es una trampa si el método reutilizado no hace lo que el nuevo contexto necesita. En este mismo proyecto, `estoy_en_la_página_de_login()` (pensado como `Given`, para **arrancar** en la página de login) navega él mismo a `login.html` en vez de comprobar dónde ha quedado el navegador. Reutilizado como `Then` tras un intento de acceso a una página protegida, el escenario "pasaría" aunque el filtro de seguridad no redirigiera a ningún sitio — el `Then` iría a `login.html` por su cuenta de todas formas, sin comprobar nada real. Antes de reutilizar un paso con una palabra clave distinta a la que se escribió, hay que revisar qué hace el método por dentro, no solo si el texto encaja.

## 6. DataTables: una tabla para un único paso

Cuando un paso necesita varias filas de datos relacionados, se le puede adjuntar una tabla directamente debajo (sin ninguna palabra clave adicional):

```
Feature: Cesta de la compra 2
  Scenario: Comprar un teclado
    Given los siguientes productos
      | nombre producto | precio | cantidad |
      | Chintáfono      | 20     | 1         |
      | Fleje            | 30     | 2         |
      | Studebaker       | 40     | 3         |

    When Compro la cesta
    Then el total es 200.0
```

GHERKIN

El step recibe un `DataTable` y decide cómo convertirlo — la forma más directa es como una lista de mapas, uno por fila, con la cabecera como claves:

```
@Given("los siguientes productos")
public void los_siguientes_productos(DataTable dataTable) {
    List<Map<String, String>> filas = dataTable.asMaps(String.class, String.class);
    for (Map<String, String> fila : filas) {
        String nombre = fila.get("nombre producto");
        Double precio = Double.valueOf(fila.get("precio"));
        Integer cantidad = Integer.valueOf(fila.get("cantidad"));
        // ...
    }
}
```

JAVA

(adaptado de `Ej12_Cucumber/src/test/java/com/proyecto/Ejemplo02.java`)



`dataTable.asMaps` siempre devuelve `String` — la conversión a `Double` / `Integer` / lo que corresponda es responsabilidad del propio step, como en el ejemplo. `DataTable` tiene otros métodos (`asList`, `asLists`) para tablas de una sola columna o sin cabecera con nombre.

## 7. Scenario Outline + Examples : repetir un escenario con datos distintos

Ninguno de los `.feature` de estos proyectos usa todavía `Scenario Outline`, pero es la herramienta correcta cuando el **mismo** escenario debería repetirse una vez por cada valor de una lista, en vez de copiarlo a mano o meter un bucle dentro de un único step.

```
Scenario Outline: Acceso a un recurso seguro sin sesión
    When intento acceder directamente a "<recurso>"
    Then estoy en la página de login
```

GHERKIN

Examples:

|                                |  |
|--------------------------------|--|
| recurso                        |  |
| /seguro/inicio.jsp             |  |
| /seguro/SVClientes             |  |
| /seguro/formularioClientes.jsp |  |

Cucumber ejecuta el escenario **una vez por fila** de `Examples`, sustituyendo `<recurso>` por el valor de esa columna antes de emparejar el texto contra el step — por eso, si el step usa `{string}`, el placeholder tiene que ir entre comillas en el texto (`"<recurso>"`), y el nombre entre `< >` tiene que coincidir exactamente con la cabecera de la columna.



**Examples:** solo es válido dentro de un `Scenario Outline`, y va **después** de los pasos, no antes. Escribirlo en un `Scenario` normal, o antes del `When` / `Then`, no es una variante rara — es Gherkin inválido y el fichero deja de parsear.



Para un escenario con muchas filas puede interesar que **todas compartan el mismo navegador** en vez de abrir uno nuevo por fila (cada fila es, para Cucumber, un escenario independiente, así que por defecto tendría su propio `WebDriver` — ver Hooks: `@Before` / `@After`). Eso ya no se resuelve con `Scenario Outline` en sí, sino combinando un `driver` estático con un tag que controle cuándo se cierra — ver la nota sobre `not @tag` en Hooks: `@Before` / `@After`.

## 8. Background : pasos comunes a toda la feature

`login_uso.feature` tiene dos escenarios que empiezan igual:

```
Scenario: Login correcto
  Given estoy en la página de login
  When introduzco mis credenciales
  ...

Scenario: Login incorrecto
  Given estoy en la página de login
  When introduzco unas credenciales incorrectas
  ...
```

GHERKIN

Un `Background` saca ese `Given` repetido a un único sitio, y Cucumber lo ejecuta automáticamente al principio de **todos** los escenarios de la feature:

```
Feature: Autenticación de usuarios

Background:
  Given estoy en la página de login

Scenario: Login correcto
  When introduzco mis credenciales
  And pulso el botón entrar
  Then se crea mi sesión
  And veo la página de inicio

Scenario: Login incorrecto
  When introduzco unas credenciales incorrectas
  And pulso el botón entrar
  And estoy en la página de login
```

GHERKIN



`Background` no ahorra tiempo de ejecución — Cucumber lo vuelve a ejecutar entero para cada escenario, solo ahorra líneas repetidas en el fichero. Y solo debería contener pasos que de verdad apliquen a **todos** los escenarios de la feature: si algún escenario no lo necesita, es señal de que ese paso no pertenece al `Background`, o de que esa feature está mezclando casos que deberían ir en ficheros distintos.

## 9. Hooks: @Before / @After

```
public class Common {  
  
    private final DriverManager driverManager;  
  
    public Common(DriverManager driverManager) {  
        this.driverManager = driverManager;  
    }  
  
    @Before("@Autenticado")  
    public void beforeEach() {  
        LoginUtil.login(driverManager.getDriver(), "aaa", "bbb");  
    }  
}
```

JAVA

(Ej12\_Cucumber\_Selenium/src/test/java/com/curso/feature/Common.java)

- Sin condición (@Before, @After a secas), un hook se ejecuta para **todos** los escenarios de **todas** las features del proyecto, no solo los de la clase donde está declarado — es un mecanismo global, no ligado a una clase de steps concreta.
- Con una expresión de tags (@Before("@Autenticado")), el hook solo se dispara para los escenarios cuya feature o escenario lleve ese tag:

```
@Autenticado  
Feature: Listado de clientes  
  
Scenario: Ver el listado con clientes existentes  
    When accedo al listado de clientes  
    Then veo una fila por cada cliente con su nombre, dirección y teléfono
```

GHERKIN



Los hooks sin condición son globales precisamente por eso: si **dos clases de steps distintas** declaran cada una su propio @Before que crea un WebDriver, Cucumber ejecuta **ambos** para cada escenario, aunque el escenario solo use los pasos de una de las dos clases — resultado, dos navegadores abiertos por un único escenario. Es justo lo que pasaba en una versión anterior de Login.java / Clientes.java de este proyecto, antes de centralizar la creación del driver en una única clase (Common / DriverManager, ver Compartir estado entre clases de steps).

Ej12\_Cucumber\_S-SDLC resuelve lo mismo con una clase dedicada solo a hooks, sin steps propios:

```
public class Hooks {  
  
    @Before  
    public void limpiarCesta() {  
        EstadoCesta.eliminarCesta();  
    }  
}
```

JAVA

## 10. Compartir estado entre clases de steps

Cucumber crea una instancia nueva de cada clase de steps **por escenario** (no reutiliza objetos entre escenarios). Cuando varias clases necesitan compartir datos dentro del mismo escenario (el producto que se acaba de añadir, el `WebDriver` abierto...), hay dos patrones en estos proyectos:

### 10.1. Campo estático compartido

```
public class EstadoCesta {  
  
    private static Cesta cesta;  
  
    public static Cesta getCesta() {  
        if (cesta == null) {  
            cesta = new Cesta();  
        }  
        return cesta;  
    }  
  
    public static void eliminarCesta() {  
        cesta = null;  
    }  
}
```

JAVA

( `Ej12_Cucumber/src/test/java/com/proyecto/EstadoCesta.java` , usado igual en `Ej12_Cucumber_S-SDLC` )

Sencillo, pero con el mismo riesgo que cualquier estado `static` : si dos escenarios se ejecutaran en paralelo, compartirían la misma `Cesta` sin darse cuenta. Por eso `Hooks.limpiarCesta()` resetea el estático en cada `@Before` — es una forma de mitigar el riesgo, no de eliminarlo.

### 10.2. Inyección de dependencias por constructor ( `cucumber-picocontainer` )

```
<dependency>  
<groupId>io.cucumber</groupId>  
<artifactId>cucumber-picocontainer</artifactId>  
<version>7.34.2</version>  
<scope>test</scope>  
</dependency>
```

XML

Con esta dependencia en el classpath, Cucumber usa `PicoContainer` como **object factory**: si varias clases de steps piden el **mismo tipo** en su constructor, reciben la **misma instancia**, creada una sola vez por escenario:

```
public class DriverManager {  
    private final WebDriver driver;  
  
    public DriverManager() {  
        System.setProperty("webdriver.gecko.driver", "...");  
        this.driver = new FirefoxDriver();  
    }  
  
    public WebDriver getDriver() { return driver; }  
}
```

JAVA

```
public class Login {
    private final WebDriver driver;

    public Login(DriverManager driverManager) {
        this.driver = driverManager.getDriver();
    }
    // ...
}
```

`Login`, `Clientes` y `Common` (`Ej12_Cucumber_Selenium`) piden todas `DriverManager` en su constructor — `PicoContainer` crea uno solo por escenario y se lo pasa a las tres, así comparten el mismo navegador sin necesidad de campos `static`.

Sin `cucumber-picocontainer` en el `pom.xml`, Cucumber usa su `DefaultObjectFactory`, que **no hace inyección de dependencias**: exige que cada clase de steps tenga un constructor público sin argumentos. En cuanto una clase de steps tiene un constructor con parámetros (como `Login` arriba) sin esa dependencia añadida, falla en tiempo de ejecución con:



```
io.cucumber.core.exception.CucumberException: class com.curso.feature.Common does not have
a public zero-argument constructor.
```

Es un error real que se reproduce fácilmente si se añade un constructor con parámetros a una clase de steps sin haber añadido antes `cucumber-picocontainer` (u otro **object factory** como `cucumber-spring`).

## 11. Casos de uso, abuso y mal uso (S-SDLC)

`Ej12_Cucumber_S-SDLC` usa Cucumber para algo más que "camino feliz": junto a los escenarios de uso normal, tiene features dedicadas a describir cómo un **atacante** o un **usuario descuidado** podrían intentar romper el sistema, con la misma estructura `Como/Quiero/Para` que cualquier otra historia de usuario, pero puesta en boca del atacante:

Feature: Abuso de la cesta de la compra

Scenario: Un atacante intenta añadir un producto con precio negativo

Given el precio de un teclado es -50.0

When Compro 1 teclados

Then la operación es rechazada

Scenario: Un atacante reutiliza el id de un producto para sustituir su precio

Given el precio de un teclado es 100.0

And Compro 1 teclados

When intento añadir un producto con el mismo id y precio 1.0

Then el total sigue siendo 100.0

Y `mal_uso_pedido.feature` cubre el caso intermedio: no hay intención maliciosa, pero un usuario legítimo puede provocar el mismo tipo de problema por accidente (doble clic en "confirmar pedido"):

Feature: Mal uso del pedido de compra

Como comprador legítimo  
 Quiero confirmar mi pedido  
 Para recibir mis productos, aunque a veces pulse "comprar" dos veces por impaciencia

Scenario: Doble envío del mismo pedido por doble clic  
 Given una cesta con un teclado de precio 100.0  
 And un pedido con identificador de envío "ENVIO-123"  
 When confirmo el pedido dos veces seguidas  
 Then solo se debe registrar un pedido

Escribir estos casos como escenarios Gherkin normales (no como comentarios en el código, ni como una lista aparte en un documento de riesgos) tiene la misma ventaja que cualquier otro uso de BDD: quedan como especificación ejecutable — si alguien rompe sin querer la validación que impide el precio negativo, el escenario falla igual que fallaría cualquier otro test, en vez de depender de que alguien recuerde revisarlo a mano.

## 12. Ejecutar los escenarios: JUnit Platform + Maven/Eclipse

Cucumber se integra con Maven/Eclipse a través de `cucumber-junit-platform-engine`, que registra Cucumber como un motor más de JUnit Platform. Para tener un punto de entrada claro hace falta una clase **runner**:

JAVA

```
package com.curso;

import org.junit.platform.suite.api.IncludeEngines;
import org.junit.platform.suite.api.SelectPackages;
import org.junit.platform.suite.api.Suite;

@Suite
@IncludeEngines("cucumber")
@SelectPackages("features")
public class RunCucumberTest {
}
```

`@SelectPackages("features")` le dice a Cucumber dónde buscar los `.feature` (`src/test/resources/features/` en este proyecto); usar `@SelectClasspathResource` en su lugar funciona, pero Cucumber avisa (con un warning, no un error) de que `@SelectPackages` es el selector recomendado para este caso.



El nombre de la clase importa más de lo que parece. **Surefire** (el plugin de Maven que ejecuta los tests) solo considera candidatas a las clases cuyo nombre sigue su convenio por defecto (`**/*Test.java`, `**/Test*.java` ...). Ninguna clase de steps normal encaja ahí — por eso hace falta una clase runner con un nombre como `RunCucumberTest`: sin ella, `mvn test` puede terminar en `BUILD SUCCESS` sin haber ejecutado ni un solo escenario, en silencio, aunque los `.feature` y los steps estén todos bien escritos.

## 12.1. Dependencias necesarias

```
<dependency>
  <groupId>org.junit.platform</groupId>
  <artifactId>junit-platform-suite</artifactId>
  <version>1.14.3</version>
  <scope>test</scope>
</dependency>
<dependency>
  <groupId>org.junit.platform</groupId>
  <artifactId>junit-platform-commons</artifactId>
  <version>1.14.3</version>
  <scope>test</scope>
</dependency>
<dependency>
  <groupId>org.junit.platform</groupId>
  <artifactId>junit-platform-launcher</artifactId>
  <version>1.14.3</version>
  <scope>test</scope>
</dependency>
```

XML

- `junit-platform-suite` — necesaria para que compilen `@Suite` / `@IncludeEngines` / `@SelectPackages` .
- `junit-platform-launcher` — la usa Eclipse (y Surefire) para descubrir qué motores de JUnit Platform hay en el classpath (incluido el de Cucumber) y lanzarlos. Sin ella, "Run As → JUnit Test" en Eclipse no encuentra nada que ejecutar aunque exista la clase runner.



`cucumber-junit-platform-engine` arrastra su propia versión de `junit-platform-engine` como dependencia transitiva, que puede no coincidir con la versión que se declara a mano para `junit-platform-suite` / `commons` / `launcher` . Si no coinciden, el fallo no es un simple warning: la ejecución revienta con `NoClassDefFoundError` (por ejemplo, `org/junit/platform/commons/io/Resource` ) porque una clase nueva de una versión no existe en la versión antigua de otro artefacto del mismo módulo. La solución es fijar **todos** los artefactos `org.junit.platform:*` a la misma versión — comprobarlo con `mvn dependency:tree` si aparece este error.

## 12.2. maven-surefire-plugin

```
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-surefire-plugin</artifactId>
  <version>3.2.5</version>
</plugin>
```

XML

Sin fijar la versión, Maven puede caer en un Surefire por defecto tan antiguo que no entiende JUnit Platform en absoluto — `mvn test` termina en `BUILD SUCCESS` habiendo ejecutado 0 tests, sin ningún error que lo delate. Con una versión moderna (3.x), Surefire detecta automáticamente que hay un motor de JUnit Platform en el classpath y activa el **provider** adecuado — no hace falta configurar nada más.

## 13. Informes legibles

El XML que genera Surefire por escenario ( `target/surefire-reports/TEST-*.xml` ) solo dice "escenario X: pasa/falla", sin detalle de qué paso concreto falló. Cucumber trae sus propios formatos de salida, configurables con la propiedad `cucumber.plugin`:

```
# src/test/resources/junit-platform.properties
cucumber.plugin=pretty, html:target/cucumber-reports/report.html
```

PROPERTIES

- `pretty` — imprime el Gherkin real en la consola, con ✓/✗ por paso y la traza justo debajo del paso que falló.
- `html:ruta` — genera un informe HTML navegable en esa ruta, con el mismo detalle.

El formatter `html` no es un plugin de Maven ni de terceros: es el artefacto oficial `io.cucumber:html-formatter`, que ya viene como dependencia transitiva de `cucumber-core` — no hace falta añadir nada al `pom.xml`, solo declarar el fichero de propiedades.

`junit-platform.properties` en el classpath funciona igual se lance desde Maven o desde Eclipse. Si se prefiriera tenerlo también en el `pom.xml`, se puede pasar como propiedad de sistema al propio Surefire:



```
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-surefire-plugin</artifactId>
  <configuration>
    <systemPropertyVariables>
      <cucumber.plugin>pretty, html:target/cucumber-reports/report.html</cucumber.plugin>
    </systemPropertyVariables>
  </configuration>
</plugin>
```

XML

pero esa vía solo se aplica cuando se ejecuta vía `mvn test` — lanzado desde Eclipse (sin pasar por Surefire) esa configuración no llegaría, así que conviene mantener también el fichero de propiedades si se quiere el mismo informe en los dos sitios.

## 14. Buenas prácticas (y errores comunes)

- **Un hook sin tag es global**, no de "su" clase — antes de añadir un `@Before` en una clase de steps nueva, comprobar si ya existe uno equivalente en otra clase; es fácil acabar con dos haciendo lo mismo por duplicado (ver Hooks: `@Before` / `@After` ).
- **Cucumber empareja por texto, no por palabra clave**. Reutilizar un paso escrito como `Given` en un `Then` (o viceversa) funciona, pero solo tiene sentido si el método realmente verifica algo en ese nuevo contexto — no si navega o modifica estado pensando que es una precondition.
- **Background solo para lo común a todos los escenarios de la feature**, y corto (la propia documentación de Cucumber recomienda 3-4 pasos como mucho) — si crece más, suele ser señal de que la feature mezcla demasiadas cosas.
- **Estado compartido static (como EstadoCesta) es simple pero frágil bajo ejecución en paralelo**; la inyección de dependencias por constructor ( `cucumber-picocontainer` ) da una instancia por escenario sin ese riesgo, al coste de un poco más de código de configuración.

- **Los casos de abuso/mal uso merecen su propio** `.feature`, con la misma estructura `Como/Quiero/Para` puesta en boca del atacante o del usuario descuidado — quedan como especificación ejecutable del riesgo, no como un comentario que nadie vuelve a mirar.
- **Fijar la versión de todos los artefactos** `org.junit.platform:*` **a la misma línea** cuando se mezclan con `cucumber-junit-platform-engine` — es la causa más probable de un `NoClassDefFoundError` que a primera vista no parece tener nada que ver con Cucumber.
- **Sin una clase runner que siga el convenio de nombres de Surefire, Maven no ejecuta nada, en silencio.** Un `BUILD SUCCESS` sospechosamente rápido, con 0 tests, es la señal de este problema.

## 15. Referencia rápida

| Elemento                                                                                               | Para qué                                                                                      |
|--------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| <code>Feature:</code> / <code>Scenario:</code>                                                         | Agrupación / caso concreto.                                                                   |
| <code>Given</code> / <code>When</code> / <code>Then</code> / <code>And</code>                          | Pasos. Cucumber empareja por texto, no por la palabra clave usada.                            |
| <code>{int}</code> / <code>{double}</code> / <code>{string}</code>                                     | Cucumber Expressions incorporadas para capturar valores en el texto del paso.                 |
| <code>DataTable</code> (tabla bajo un paso)                                                            | Varias filas de datos relacionados para un único step.                                        |
| <code>Scenario Outline</code> + <code>Examples:</code>                                                 | Repite el escenario una vez por fila, sustituyendo <code>&lt;columna&gt;</code> en los pasos. |
| <code>Background:</code>                                                                               | Pasos comunes a todos los escenarios de la feature.                                           |
| <code>@Before</code> / <code>@After</code>                                                             | Hooks globales, salvo que se les dé una expresión de tags ( <code>@Before("@tag")</code> ).   |
| <code>@tag</code> sobre <code>Feature:</code> / <code>Scenario:</code>                                 | Etiqueta para filtrar ejecución o condicionar hooks.                                          |
| <code>cucumber-picocontainer</code>                                                                    | Inyección de dependencias por constructor entre clases de steps.                              |
| <code>@Suite</code><br><code>@IncludeEngines("cucumber")</code><br><code>@SelectPackages("...")</code> | Clase runner para que Maven/Eclipse descubran y ejecuten los <code>.feature</code> .          |
| <code>cucumber.plugin=pretty,</code><br><code>html:ruta</code>                                         | Informe legible en consola y en HTML ( <code>junit-platform.properties</code> ).              |



Todos los conceptos de este manual se pueden practicar de forma guiada en las prácticas `Ej12_Cucumber`, `Ej12_Cucumber_S-SDLC` y `Ej12_Cucumber_Selenium` del curso.